SWE 4101

Object Oriented Concepts I

Prepared By:
Samia Nawsheen
Junior Lecturer, CSE

Objectives

01 Recap: interfaces as contracts

02 The evolution problem

03 Default, static & private methods

04 The diamond problem, reborn

05 Functional interfaces & lambdas

06 Marker interfaces & abstraction

Recap

- Interface = a contract: signatures, no state, no constructor
- implements creates a subtype without inheriting any code
- One class can implement many interfaces; extend only one class
- Interfaces group by ability, not identity

The Evolution Problem

- Hundreds of classes already implement Payable
- We now want every payable to report a receipt line
- Adding a new abstract method breaks every existing implementer
- We need a way to grow the contract without breaking anyone

The Break, Illustrated

```
interface Payable {
    long amountOwedThisMonth();
    String receiptLine(); // NEW - breaks every implementer!
}
```

- Every existing class instantly fails to compile
- This was a real problem when Java 8 added lambdas to Collection

Default Methods

- Starting from Java 8, interfaces may have default methods
- A method with a body, supplied by the interface itself
- Every implementer receives it for free
- Existing classes keep compiling unchanged
- Any class may still override it

Default Methods in Practice

```
interface Payable {
    long amountOwedThisMonth();

    default String receiptLine() {
        return "Owed: " + amountOwedThisMonth();
    }
}
```

- receiptLine() is defined purely in terms of the abstract method
- Old implementers compile unchanged and get it for free
- A class wanting a nicer receipt simply overrides it

Common mistake

- Treating default methods as license for ordinary shared code
- A default method still can't touch instance variables
- Real shared state still belongs in a class, not an interface

Question for the class

Payable and Refundable each declare a default note() with a different body. A class implements both. When you call note(), whose default runs — and how could the compiler know?

Working it through

- It cannot know — Java refuses to guess
- The diamond ambiguity resurfaces, but only because both defaults have bodies
- The implementing class must override note() itself
- It may delegate explicitly: Payable.super.note()

Resolving the Conflict

```
class Contractor implements Payable, Refundable {
    public String note() {
        return Payable.super.note(); // must choose explicitly
    }
}
```

- Pure abstract signatures of the same name are harmless
- Only method bodies create real ambiguity

Static & Private Methods

- Static methods are utilities tied to the interface itself, called on the interface name
- e.g. Math.max(...)
- Private methods share code between default methods internally

Static & Private Methods

```
interface Flyable {
    static int mphToKmh(int mph) {
        return (int) (mph * 1.609);
    }

    default void takeOff() {
        prepareEngine();           // uses the private helper
    }

    private void prepareEngine() {
        System.out.println("Checking engine");
    }
}
```

Interface Constants

```
public interface Payable {
    long MIN_PAYOUT_PAISA = 10000;
    long amountOwedThisMonth();
}
```

- Any field written in an interface is implicitly public static final
- A shared constant, no per-object state
- Referred to as Payable.MIN_PAYOUT_PAISA anywhere

Functional Interfaces

- An interface with exactly one abstract method
- default / static / private methods don't count toward that one
- @FunctionalInterface asks the compiler to enforce it

```
@FunctionalInterface
interface PayRule {
    long compute(long basePaisa);
}
```

Interfaces Can Extend Interfaces

```
interface Settlement extends Payable, Auditable {
    String settlementReference();
}
```

- Interfaces use extends on each other, not implements
- One interface can extend several others — no state to collide
- Implementing Settlement means fulfilling every parent's promise too

Abstraction through Interfaces

- Separates what something does from how it does it
- Code that uses a capability never depends on who provides it
- This leads to maintainable design

Program to the Interface

```
interface LedgerSink {
    void write(String line);
}

void settle(Payable[] payees, LedgerSink sink) { ... }
```

- The engine depends only on the contract
- It has never heard of any concrete sink

Swappable Implementations

```
class ConsoleSink implements LedgerSink {
    public void write(String line) {
        System.out.println(line);
    }
}

run.settle(payees, new ConsoleSink());
run.settle(payees, new FileSink());
```

- Same engine, different behavior chosen by what you hand in
- Not one line of PayrollRun changes between the two calls

Benefits

- The user depends only on the promise, never a concrete detail
- We can replace one implementation with another without changing the code that uses it
- Different developers can work on different implementations at the same time

Practice exercise

1. Create two interfaces with a conflicting default method
2. Resolve the conflict with Interface.super.method()

Thank You